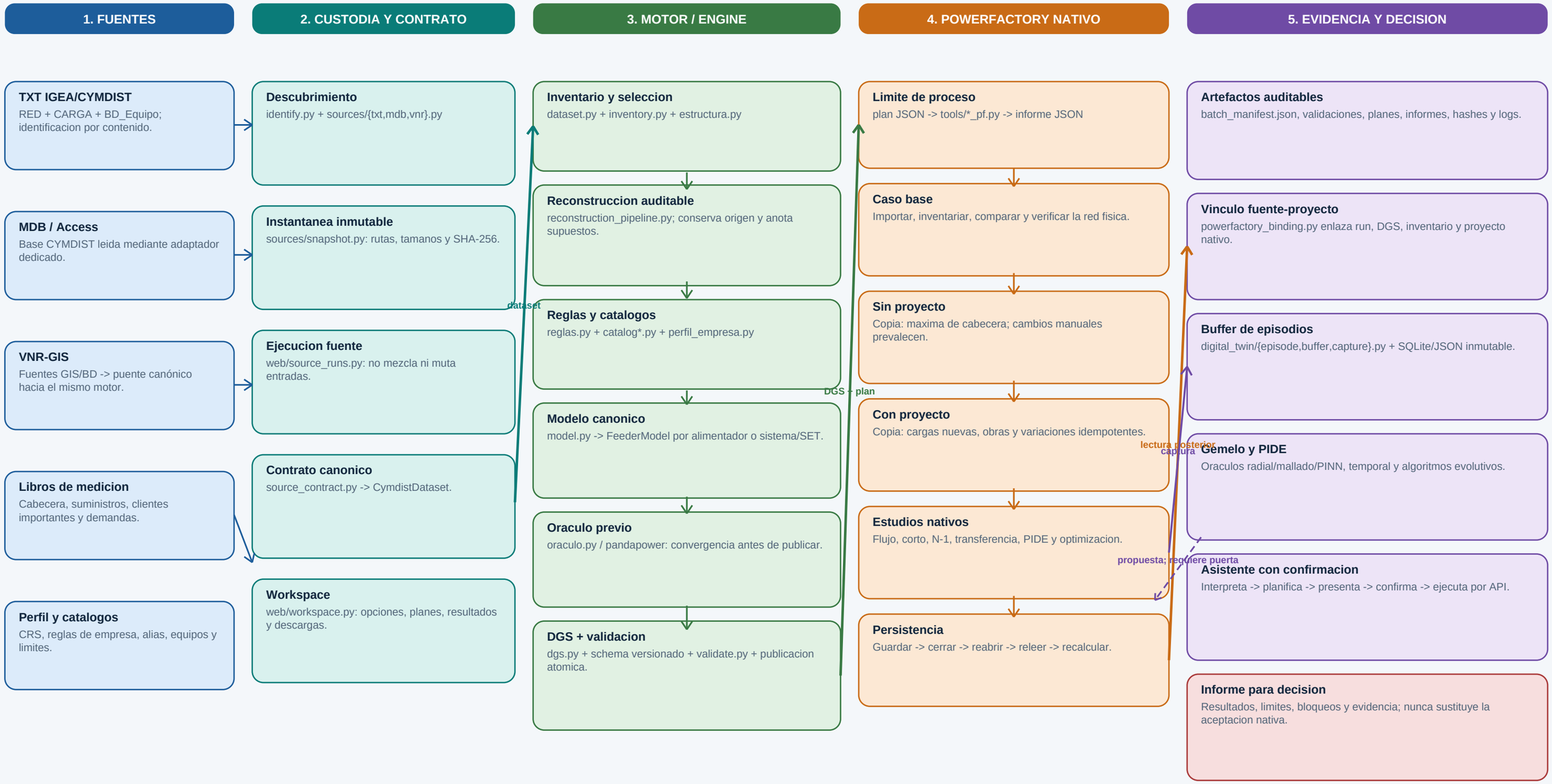
Plano maestro: arquitectura y flujo por cajas

Vista AS-IS basada en el codigo del commit f0478d6 (2026-10-25). Flecha continua = datos; punteada = control o realimentacion.



Carriles serializados de web/jobs.py

engine | powerfactory | asistente

Cada carril tiene un hilo; PowerFactory conserva exclusion mutua. El asistente orquesta y espera; no escribe por fuera de las APIs.

Flujo de trabajo desde cero: secuencia operativa

Cada caja produce evidencia verificable. Una caja roja es una puerta de bloqueo; no se salta por exito de interfaz o HTTP 200.

1 Definir alcance
Empresa, fuente, CRS, periodo, alimentadores o sistema/SET.

2 Ingresar fuente
TXT, MDB o VNR-GIS; libros de demanda como fuentes auxiliares.

3 Identificar contenido
Reconocer tablas y roles por contenido, no por nombre de archivo.

4 Custodiar ejecucion
Snapshot inmutable + SHA-256 + run_id; prohibido mezclar modalidades.

5 Cargar una vez
Adaptador -> contrato canonico -> CymdistDataset indexado.

6 Inventariar
Alimentadores, SET, tablas, cobertura de catalogo y faltantes.

7 Seleccionar
Uno, varios, todos o sistema por SET; resolver alcance esperado.

8 Reconstruir copia
Completar faltantes con jerarquia auditable; conservar valor original.

9 Aplicar reglas
Perfil de empresa, catalogos, coordenadas, puentes y transformadores.

10 Construir modelo
FeederModel canonico, estricto y determinista.

11 Preflight electrico
Oraculo independiente; registrar convergencia o bloqueo.

12 Escribir DGS
Perfil versionado, FID, cubiculos, grafico, GPS y tablas.

13 Validar y publicar
Completitud + hash + publicacion atomica + batch_manifest.

14 Importar en PF
Plan JSON; crear/verificar Caso base en proceso nativo aislado.

15 Aceptar Caso base
Releer inventario, topologia, GPS, tipos, tensiones y flujo.

16 Calibrar
Copiar a Sin proyecto; ajustar maxima de cabecera y preservar manual.

17 Integrar proyecto
Copiar a Con proyecto; cargas y obras en variaciones idempotentes.

18 Ejecutar estudios
Flujo, corto, N-1, capacidad, transferencia y alternativas.

19 Persistir y reabrir
Guardar, cerrar, nueva sesion, releer y recalcular.

20 Capturar episodio
Plan + resultado + snapshot + limites -> JSON/SQLite inmutable.

21 Evaluar / aprender
Gemelo, temporal, PIDE y evolutivos proponen; puertas comparan.

22 Decidir e informar
Aceptar/rechazar con trazabilidad; emitir informe y artefactos.

Cadena minima de evidencia

fuente + hash → run manifest → modelo → DGS + validacion → plan PF → informe PF → reread → binding → episodio → informe

Arquitectura logica, contratos y puertas de aceptacion

Responsabilidades observadas en el repositorio actual. Las capas son logicas; la migracion fisica de la spec 002 sigue siendo un objetivo separado.

Capas de responsabilidad

Presentacion
frontend/ React + TypeScript | cli.py | web/app.py

Orquestacion
web/services.py | web/jobs.py | workspace.py | source_runs.py

Aplicacion
batch.py | combine.py | reconstruction_pipeline.py | inventory.py | estudios

Dominio
model.py | reglas.py | naming.py | geografia | catalogos | estructura

Puertos y contratos
source_contract.py | sources/base.py | schema.py | native_action.py | binding

Adaptadores
sources/txt,mdb,vnr | dgs.py | tools/*_pf.py | informes | preview

Infraestructura y artefactos
filesystem | JSON | SQLite | Access | pandapower | PowerFactory 2024

Contratos entre procesos

Fuente -> motor
CymdistDataset + snapshot + source_run; una lectura, indices reutilizables.

Motor -> salida
FeederModel -> DGS + validation + manifest; publicacion atomica.

Web -> PowerFactory
Plan JSON versionado; proceso separado; cancel file; salida JSON.

PowerFactory -> custodia
Inventario releido + binding + estado de Caso base.

Accion -> gemelo
Plan + informe + snapshot + limites + hashes = Episodio.

Asistente -> operacion
Orden -> plan visible -> confirmacion -> llamadas API existentes.

Puertas independientes - fail closed

G0 Fuente
Identidad, hash, periodo, alcance y no mezcla.

G1 Estructura
Conteos, topologia, parametros y completitud.

G2 DGS
Esquema, FID, referencias y publicacion integra.

G3 Convergencia
Oraculo y luego solver nativo; resultado explicito.

G4 Calidad electrica
Tension, carga, perdidas, corto y limites aplicables.

G5 Persistencia
Guardar, cerrar, reabrir, releer y recalcular.

G6 Produccion
Cobertura real, regresion completa y evidencia vigente.

Tres carriles actuales

- engine: lectura, conversion y auditoria.
- powerfactory: toda sesion nativa, en serie.
- asistente: coordina pasos y confirmacion.

Separacion critica

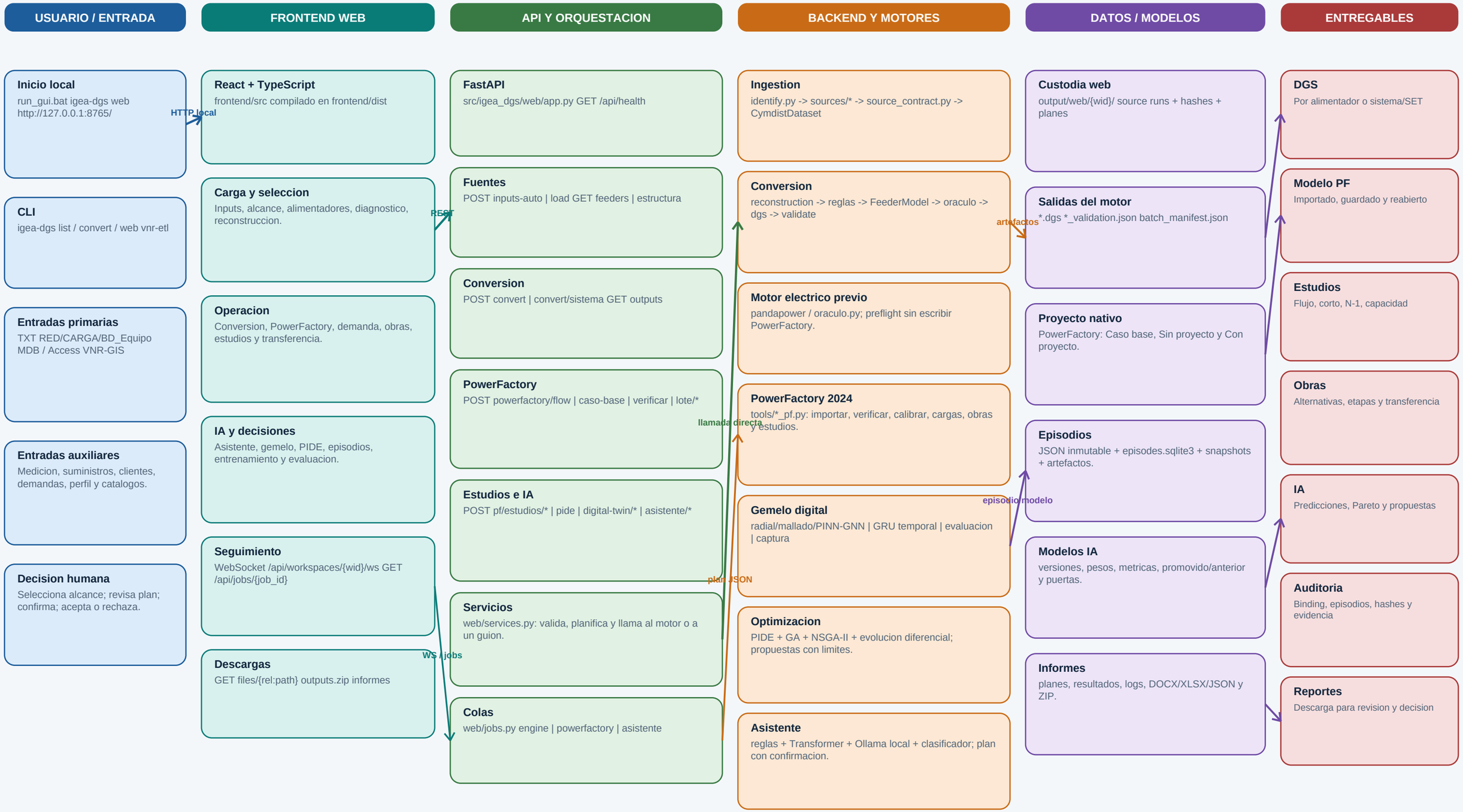
- HTTP 200, DGS valido y flujo convergente son evidencias distintas.
- El gemelo propone; PowerFactory y las puertas electricas aceptan.

Estado del plano

- Describe componentes observados; no certifica que cada spec este cerrada.
- La aceptacion depende de ejecuciones reales y artefactos vigentes.

Acceso directo: web, frontend, backend, IA y entregables

Mapa extremo a extremo con puntos de entrada reales, rutas API representativas, modulos consumidores, almacenes y salidas.



Acceso directo real: la web sirve el frontend y la API en 127.0.0.1:8765; el backend reutiliza el mismo motor que el CLI.

Limite: no hay autenticacion para exposicion publica. IA y asistente no omiten confirmacion, validacion electrica, persistencia ni aceptacion humana.

Justificacion cientifica y trazabilidad del plano

La bibliografia justifica decisiones arquitectonicas; no sustituye la evidencia del repositorio ni la validacion electrica del caso real.

Decision del plano	Base cientifica	Aplicacion concreta en el proyecto
Contrato canonico y adaptadores	[M1] propone interoperabilidad semantica para sistemas de distribucion mediante un modelo comun y mapeos normalizados. [A3] deriva intercambio CIM basado en modelos.	sources/* normaliza TXT, MDB y VNR-GIS hacia CymdistDataset; source_contract.py estabiliza el limite.
Flujo conservador y trazable	[D1] demuestra ejecuciones conservadoras de workflows heterogeneos con trazabilidad producida durante la ejecucion.	Snapshots, hashes, run_id, manifests y bindings conectan fuente, DGS, proyecto y resultado.
Oraculo abierto antes del motor propietario	[A1] documenta pandapower para automatizacion de flujo, topologia y analisis; sus modelos fueron contrastados con herramientas industriales.	oraculo.py sirve de preflight. Su exito no reemplaza la aceptacion nativa en PowerFactory.
Validacion cruzada en distribucion	[A2] valida formulaciones de flujo de distribucion contra OpenDSS y alimentadores IEEE; muestra la necesidad de comparar implementaciones y casos de referencia.	Se separan esquema, convergencia, calidad electrica y persistencia/reapertura como puertas distintas.
Separar motor y sesion nativa	[D1] y [A3] respaldan limites explicitos entre herramientas heterogeneas y contratos reproducibles.	La web pasa plan JSON a tools/*_pf.py y recibe informe JSON; PowerFactory se serializa en su carril.
Aprendizaje con evidencia, no autoridad	La trazabilidad de [D1] y la validacion comparativa de [A2] exigen conservar origen y criterio de aceptacion.	El gemelo/PIDE produce propuestas; no promueve por si solo una modificacion ni una aceptacion electrica.

Referencias verificadas en repositorios editoriales e institucionales

[M1] Tesis de maestria
Zambrano Asanza, S. P. (2011). Analisis del modelo comun de datos electrico para la integracion de sistema de manejo de la distribucion mediante estandares internacionales. Universidad de Cuenca.
https://dspace.ucuenca.edu.ec/handle/123456789/2552

[D1] Tesis doctoral
Sanchez Pina, B. A. (2021). Conservative and traceable executions of heterogeneous model management workflows. University of York.
https://etheses.whiterose.ac.uk/id/eprint/30114/

[A1] Articulo indexado
Thurner, L., et al. (2018). pandapower - An Open-Source Python Tool for Convenient Modeling, Analysis, and Optimization of Electric Power Systems. IEEE Transactions on Power Systems, 33(6), 6510-6521. DOI: 10.1109/TPWRS.2018.2829021.
https://doi.org/10.1109/TPWRS.2018.2829021

[A2] Articulo indexado
Fobes, D. M., Claeys, S., Geth, F., & Coffrin, C. (2020). PowerModelsDistribution.jl: An open-source framework for exploring distribution power flow formulations. Electric Power Systems Research, 189, 106664. DOI: 10.1016/j.epsr.2020.106664.
https://doi.org/10.1016/j.epsr.2020.106664

[A3] Articulo indexado
Uslar, M., et al. (2018). Towards model-driven CIM-based data exchange for DSOs. Energy Informatics, 1, 23.
https://doi.org/10.1186/s42162-018-0032-4

Base local revisada: docs/ARQUITECTURA.md, README.md, pyproject.toml, CODIGO_FUENTE.md, src/igea_dgs, src/vnr_etl, tools y specs.
Corte: 2026-10-25 | commit f0478d6 | alcance: arquitectura y flujo AS-IS; no constituye certificado de produccion ni aceptacion electrica universal.